

Inteligencia Artificial

Machine Learning - NLP

N-Grammars, Transformers, Modelos Pre-entrenados



Procesamiento de Lenguaje Natural (NLP)

NLP es el área de la Inteligencia Artificial que busca permitir que una computadora **entienda, interprete, genere y manipule lenguaje humano.**

Algunos desafíos:

- **Ambiguo** (una frase puede tener múltiples significados)
- **Dependiente del contexto**
- **No siempre lógico** (chistes, ironía, emociones, etc)

NLP - Un poco de historia

Período	Enfoque	Características
1950–1980	Gramáticas formales y reglas	Se intenta describir el lenguaje mediante reglas sintácticas (Chomsky, Jerarquía de Gramáticas, parsing determinístico).
1980–2010	Modelos estadísticos	Se usan corpus y probabilidad: n-grams, modelos ocultos de Markov, etiquetado POS con probabilidad.
2010–2017	Aprendizaje profundo neuronal	Modelos RNN, LSTM para secuencias; embeddings como Word2Vec y GloVe.
2017...	Transformers y modelos preentrenados	BERT, GPT, T5. Modelos capaces de comprender contexto global y generar lenguaje fluido.

Lenguajes y Gramáticas (formales)

Noam Chomsky introduce las gramáticas formales, una manera de definir estructuras del lenguaje mediante reglas.

Tipo	Nombre	Poder Expresivo	Uso típico
0	Gramáticas irrestrictas	Máxima	Modelos teóricos del lenguaje
1	Sensibles al contexto	Alta	Descripciones lingüísticas avanzadas
2	Libres de contexto (CFG)	Media	Parsers, análisis sintáctico en programación y NLP clásico
3	Regulares	Baja	Expresiones regulares, reconocimiento simple de patrones

Lenguajes y Gramáticas (formales)

Las gramáticas en el contexto de lenguajes formales son un instrumento que permiten describir la sintaxis de las palabras de un lenguaje:

- Sobre un alfabeto predefinido
- Mediante reglas (o producciones), que describen cómo se combinan los elementos del alfabeto

```
expr → term { + term }  
term → factor { * factor }  
factor → ( expr ) | number  
number → digit { digit }  
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```


Gramáticas formales - Limitaciones

Aunque las gramáticas formales son muy útiles para caracterizar la **sintaxis de un lenguaje**, su naturaleza es intrínsecamente **binaria**. Esto es, una gramática solo permite decidir si una cadena (oración) es válida (se **acepta**) o no (se **rechaza**) dentro del lenguaje que describe.

Esto característica es una **limitante** a la hora de abordar preguntas que involucren algún grado de **probabilidad o elección**, por ejemplo: qué palabra es más probable que siga a una frase parcialmente construida o cómo puedo describir una oración con un significado similar.

Para responder este tipo de preguntas, se requiere incorporar **modelos probabilísticos**, no solo reglas sintácticas.

NLP - Modelos de Lenguajes

Un **modelo de lenguaje (LM)** es un modelo que asigna probabilidad a secuencias de palabras. El principal objetivo principal es poder responder responder:

- ¿Qué tan probable es una oración?
- ¿Qué palabra es más probable que siga?

Pasa de la visión binaria de las gramáticas (acepta / rechaza) a una visión probabilística del lenguaje. Estima $P(w_1, w_2, \dots, w_n)$, a **probabilidad conjunta** de una secuencia de palabras $w_1, w_2, \dots, w_n$. También permite **predecir** la siguiente palabra: $P(w_{n+1} \mid w_1, \dots, w_n)$

NLP - Aplicaciones de Modelos de Lenguajes

Si bien hoy en día, gracias al mayor poder de cómputo y a la disponibilidad de grandes conjuntos de datos, los modelos de lenguaje tienen aplicaciones muy amplias, sus usos se pueden agrupar en dos grandes áreas:

Comprensión del Lenguaje :

- Análisis de sentimiento (opiniones en redes, reseñas)
- Clasificación de textos (spam/no spam, categorías de documentos)
- Extracción de información (entidades, relaciones, etc)
- Búsqueda semántica (motores de búsqueda más precisos)
- Asistentes de diálogo y chatbots (interpretación del usuario)

Generación y producción de Lenguaje :

- Autocompletado y corrección gramatical
- Traducción automática
- Resumen automático de documentos
- Generación de texto coherente
- Síntesis de voz y guiones para asistentes virtuales

NLP - N-Gramas

Un modelo **n-grama estima la probabilidad** de una palabra en función de las palabras anteriores, utilizando N-1 palabras como contexto previo para estimar la probabilidad de la palabra siguiente. Se basa en la Suposición de Markov: el comportamiento futuro de un sistema dinámico depende de su historia reciente.

$$P(w_n \mid w_1, w_2, \dots, w_{n-1}) \approx P(w_n \mid w_{n-(n-1)}, \dots, w_{n-1})$$

$$P(\text{"frase"} \mid \text{"cual es la palabra que le sigue a esta"})$$

NLP - Unigramas

En modelos basados en **unigramas**, la probabilidad de una sentencia se aproxima a través del producto de las probabilidades de sus palabras. En este caso no tiene en cuenta el contexto previo. Se usa la frecuencia relativa de cada palabra en el *corpus* (entrenamiento):

$$P(w) = \frac{\text{Count}(w)}{\text{Total de palabras en el corpus}}$$

Ejemplo: "el gato duerme el perro duerme": ["el": 2, "gato": 1, "perro": 1, "duerme": 2]

Como no hay contexto, se multiplican las probabilidades de cada palabra:

$$P(w_1, w_2, \dots, w_n) = \prod_i P(w_i)$$

$$P(\text{"el gato duerme"}) = (2/6) \cdot (1/6) \cdot (2/6)$$

NLP - Bigramas

En modelos basados en **bigramas**, la probabilidad de una sentencia se aproxima a través del producto de las probabilidades de **pares** de palabras. Frecuencia relativa de secuencias de palabras de tamaño 2.

El contexto es sólo la palabra previa:

$$P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

Ejemplo: ["el gato duerme", "el perro duerme", "el gato come": ["el gato": 2, "el perro": 1, "gato duerme": 1]

$$P(\text{"gato"} \mid \text{"el"}) = \frac{2}{3}, \quad P(\text{"perro"} \mid \text{"el"}) = \frac{1}{3}$$

NLP - Limitaciones de N-Gramas

Si bien son modelos muy simples y fáciles de implementar, su aplicación es muy limitada, por ejemplo para Autocompletado temprano (T9, teclados predictivos)

- No modelan contexto largo
- Crecen mucho en memoria (por ejemplo, para vocabulario de 50.000 palabras, los bigramas posibles son 50.000^2)
- Necesitan técnicas de suavizado para ser robustos

NLP - N-Gramas en Python (NLTK)

NLTK (Natural Language ToolKit) es una biblioteca Python con implementaciones de modelos de lenguajes y otros algoritmos y estructuras de datos para lenguaje natural.

El uso de NLTK facilita la implementación de modelos de lenguajes y sus aplicaciones.

Necesitamos convertir el texto en "tokens"

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

nltk.download('punkt_tab')
texto = "El gato duerme en la alfombra. El perro corre rápido."
tokens = word_tokenize(texto.lower()) # Tokenización y minúsculas

bigramas = list(ngrams(tokens, 2))
print("Bigrams:", bigramas)
```


NLP - Lenguajes como secuencia de palabras y RNN

Desvanecimiento del Gradiente: Durante el entrenamiento, el gradiente se propaga hacia atrás (BPTT). En secuencias largas los gradientes se vuelven muy pequeños.

Explosión del Gradiente: Otra posibilidad es que los gradientes se vuelvan enormes tornando el entrenamiento es inestable.

Memoria limitada: El estado oculto resume toda la información de la secuencia en un solo vector. En general las RNN recuerdan lo reciente y olvidan lo anterior. Si bien, las extensiones como LSTM y GRU mitigan estos defectos, las RNN son **sensibles al largo de la secuencia**, con secuencias cortas funcionan medianamente bien, no con secuencias largas.

Difícil optimización: Debido al inherente procesamiento secuencial, palabra por palabra siguiendo su orden, **no permiten utilizar técnicas de paralelismo** para optimizar su entrenamiento.

NLP - Modelos Neuronales de Lenguajes

Para poder procesar lenguajes naturales, necesitamos una representación para las palabras y frases. Una primera aproximación que utilizamos para RNN fue la de "one-hot vectors":

Dado un corpus (data set) de n palabras (*tokens*) $w_1, w_2, \dots, w_n$ diferentes, la representación mediante *one hot vectors* utiliza **vectores de bits de longitud n** para representar cada palabra/token:

w_1 se representa con $[1, 0, \dots, 0]$

w_2 se representa con $[0, 1, \dots, 0]$

...

w_n se representa con $[0, 0, \dots, 1]$

Mientras más grande es el "diccionario", más extensa la representación de cada palabra.

NLP - Tokenización

Una primera tarea para abordar y decidir, es cómo partimos el corpus en tokens. Este proceso se denomina **tokenización**. Debemos tener en cuenta diferentes aspectos: Mayúsculas/minúsculas, signos de puntuación, escrituras logográficas (por ejemplo chino, emojis), palabras compuestas, etc.

De manera general podemos considerar 3 tipos de tokenización:

Por palabras: es la más intuitiva, aunque puede generar diccionarios muy extensos.

Por caracteres: genera diccionarios escuetos, pero es más costoso el entrenamiento para generalizar y aprender.

Por sub-palabras: busca un equilibrio entre los anteriores. Son los más utilizados actualmente, existen alternativas como Byte Pair Encoding (BPE), WordPiece y SentencePiece

NLP - Tokenización - Byte Pair Encoding (BPE)

La idea es partir de caracteres individuales e ir combinando los pares más frecuentes para formar sub-palabras.

- Se tokeniza inicialmente en caracteres.
- Se cuentan todas las parejas de símbolos.
- Se **fusiona la pareja más frecuente** en una nueva unidad.
- Se repite hasta obtener el tamaño de vocabulario deseado.

Tiene buen balance entre granularidad y cobertura aunque podría producir tokens inválidos. Usado en GPT-2, GPT-3, GPT-4, LLaMA.

NLP - Tokenización - WordPiece

La idea es similar a BPE pero no fusiona por frecuencia sino por la probabilidad basada en un modelo.

- Se elige el merge que mejora más la probabilidad del corpus bajo un modelo.
- No solo repite el par más frecuente, sino el que mejor explica el texto.
- Utiliza una representación típica con prefijo para las subwords añadidas: "play", "##ing", "##er".

Mejora la generalización, sobre todo, en lenguajes ricos morfológicamente.
Usado en BERT.

NLP - Tokenización - SentencePiece

No depende de espacios. Opera directamente sobre la secuencia de bytes o caracteres Unicode.

- De manera contraria a los anteriores comienza con un vocabulario inicial grande y se van eliminando los tokens que menos contribuyen a la probabilidad global del texto.

Esta técnica no requiere tokenización previa, aunque su implementación es más compleja.

Texto	Método	Tokenización
"computadoras trabajando"	BPE	"compu" "tadoras" "trabaj" "ando"
	WordPiece	"computadora" "##s" "trabaj" "##ando"
	SentencePiece	"_computadoras" "_trabaj" "ando"

NLP - Dando semántica a los tokens

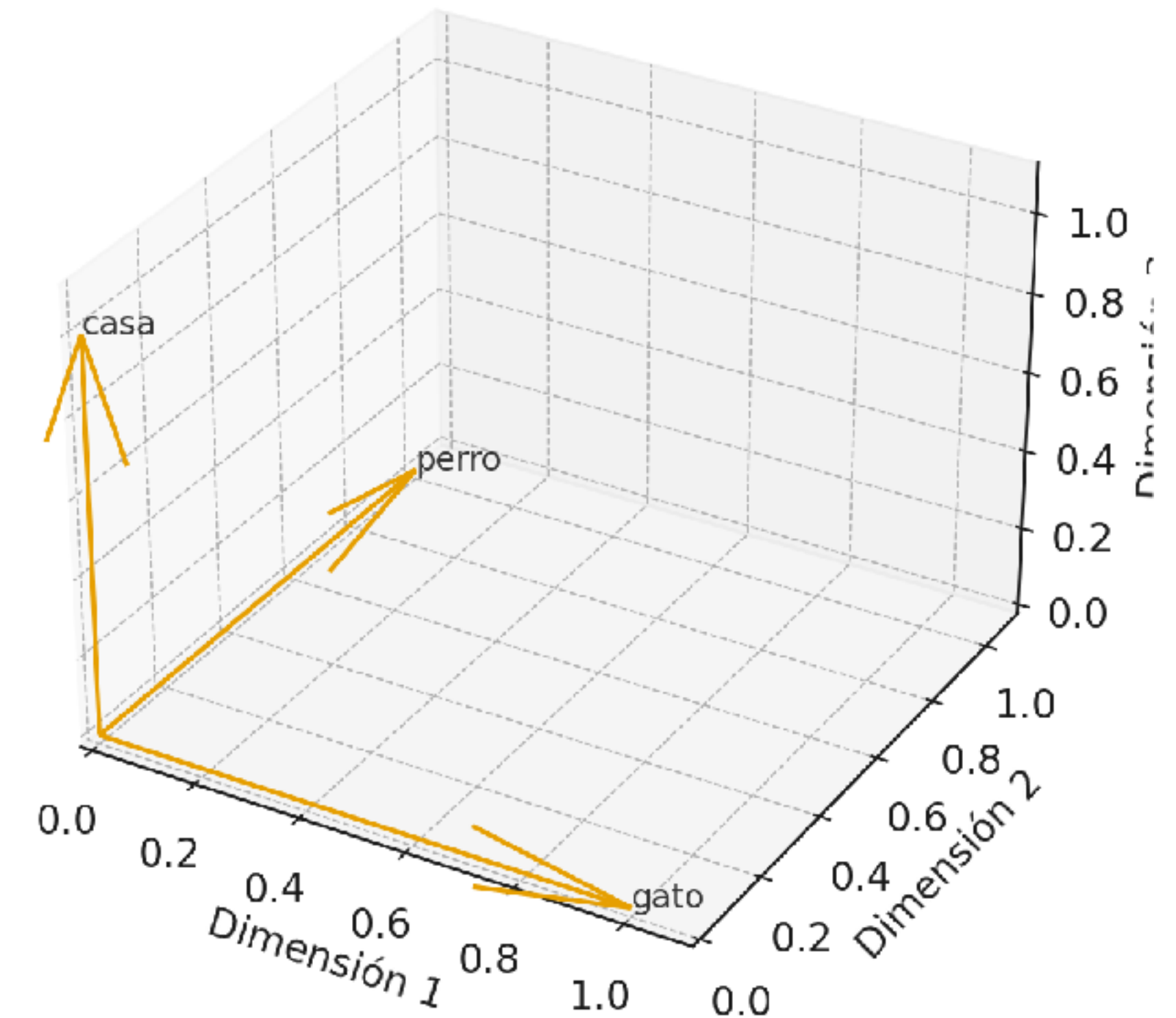
Si bien la representación de tokens como *one hot vectors* tiene diversas aplicaciones prácticas, en el contexto de NLP, esta representación sufre de ausencia de relación matemática entre las diferentes palabras, es decir, no captura ningún significado semántico entre las mismas.

Si analizamos los vectores como una representación en el hiperespacio, el **producto escalar entre dos tokens siempre es 0** (conviven en espacio ortogonalmente diferentes):

"casa": [1,0,0]

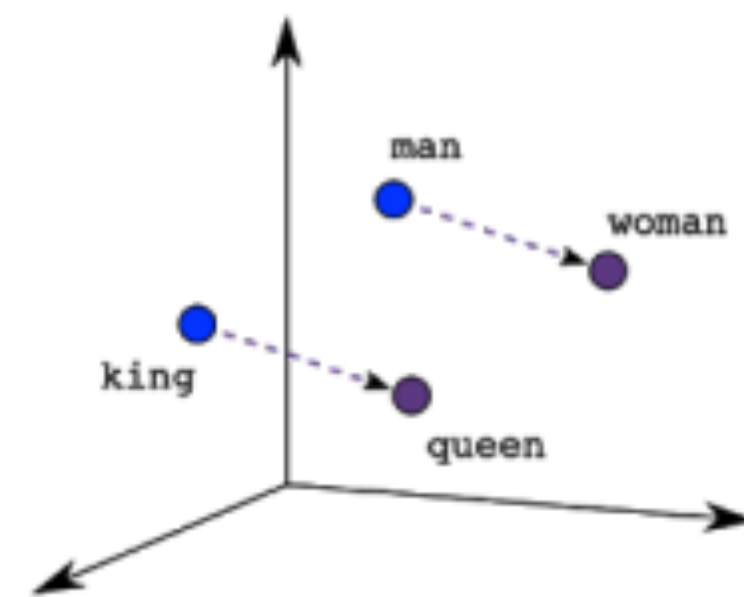
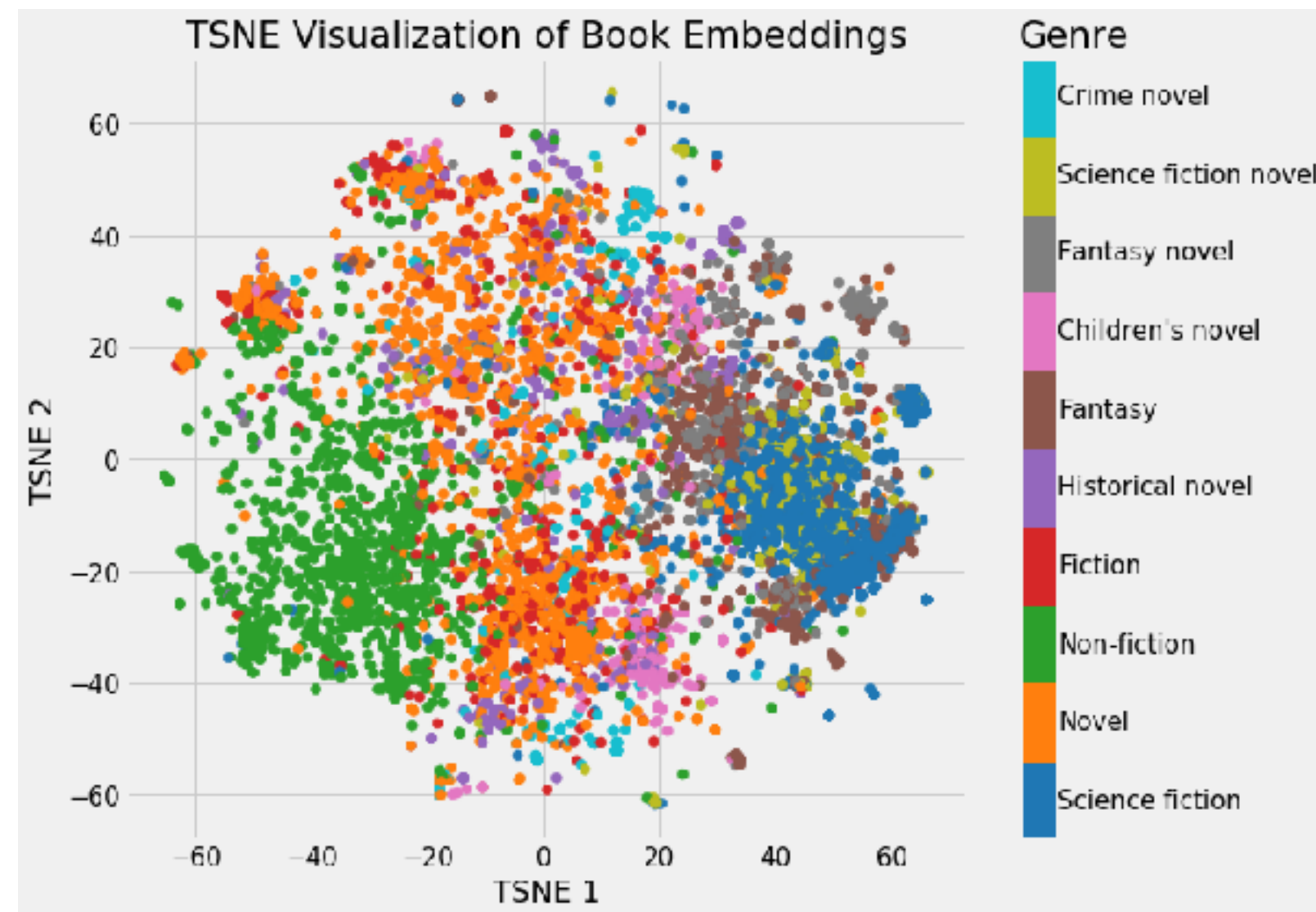
"perro": [0,1,0]

"gato": [0,0,1]

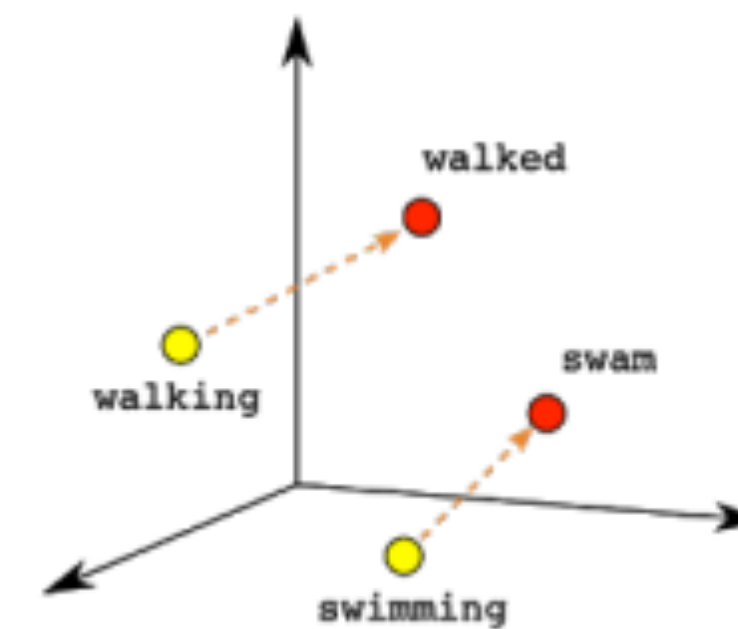


NLP - Dando semántica a los tokens = Embeddings

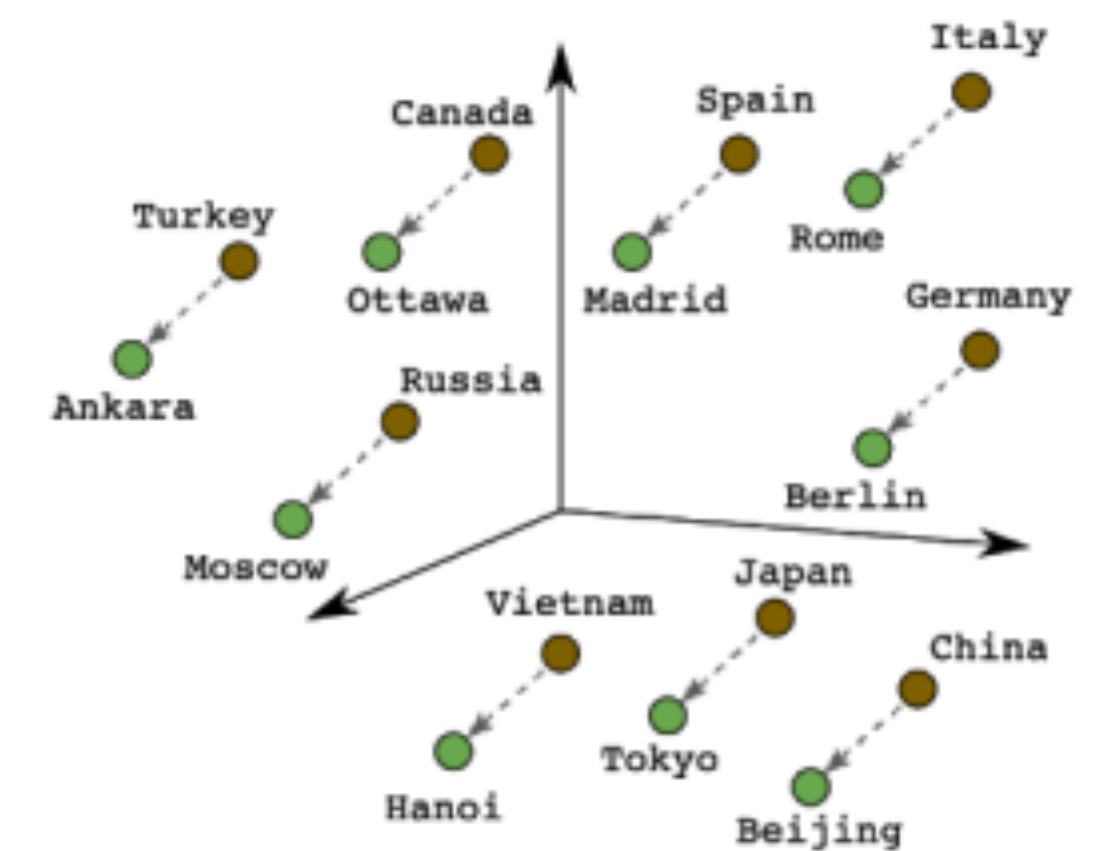
Una alternativa es proyectar los tokens en un **espacio vectorial continuo** donde las palabras con significados similares se **encuentren ubicada cerca en el espacio vectorial**. Por ejemplo, perro y gato deberían estar próximos entre sí y ambas alejadas de casa.



Male-Female



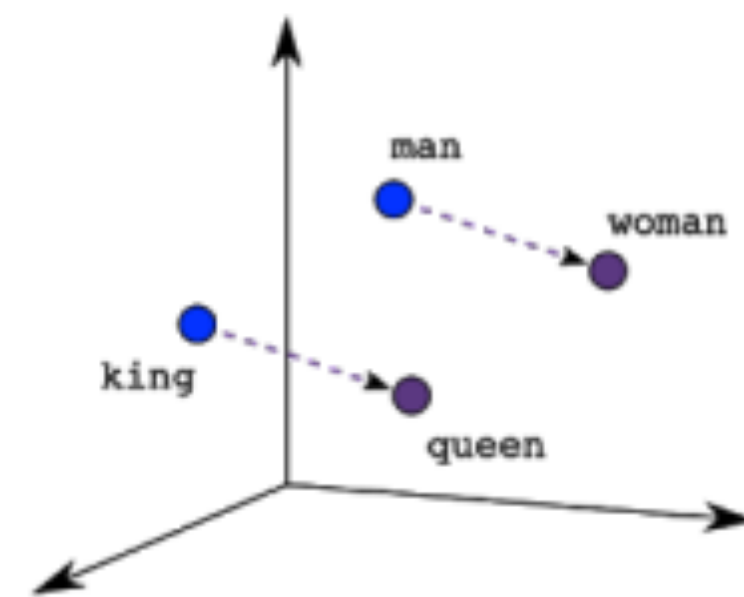
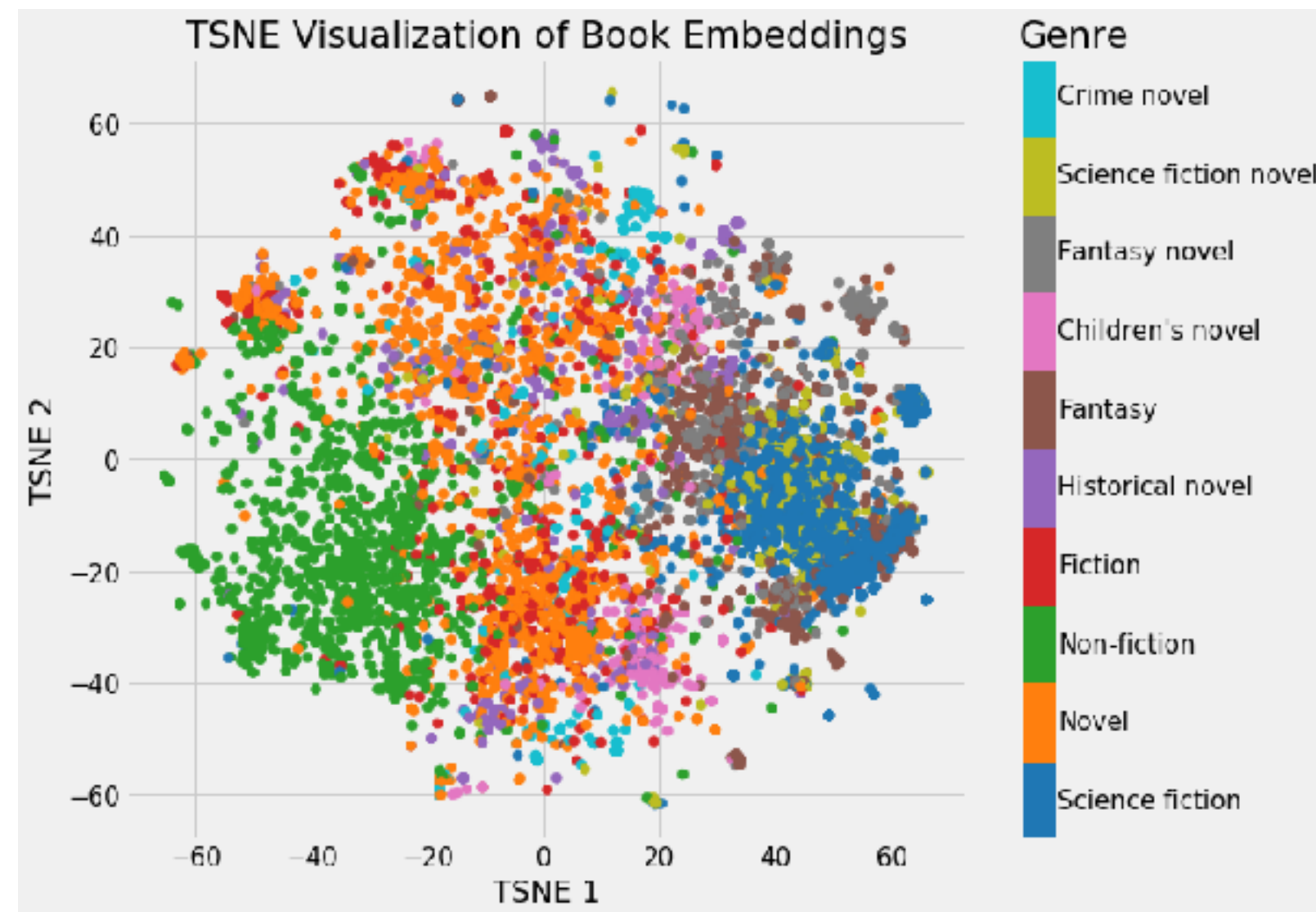
Verb Tense



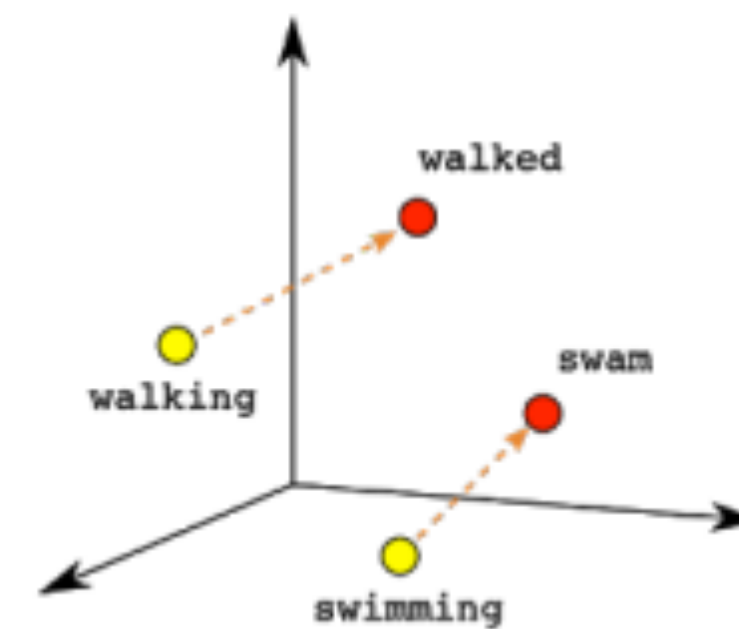
Country-Capital

NLP - Dando semántica a los tokens = Embeddings

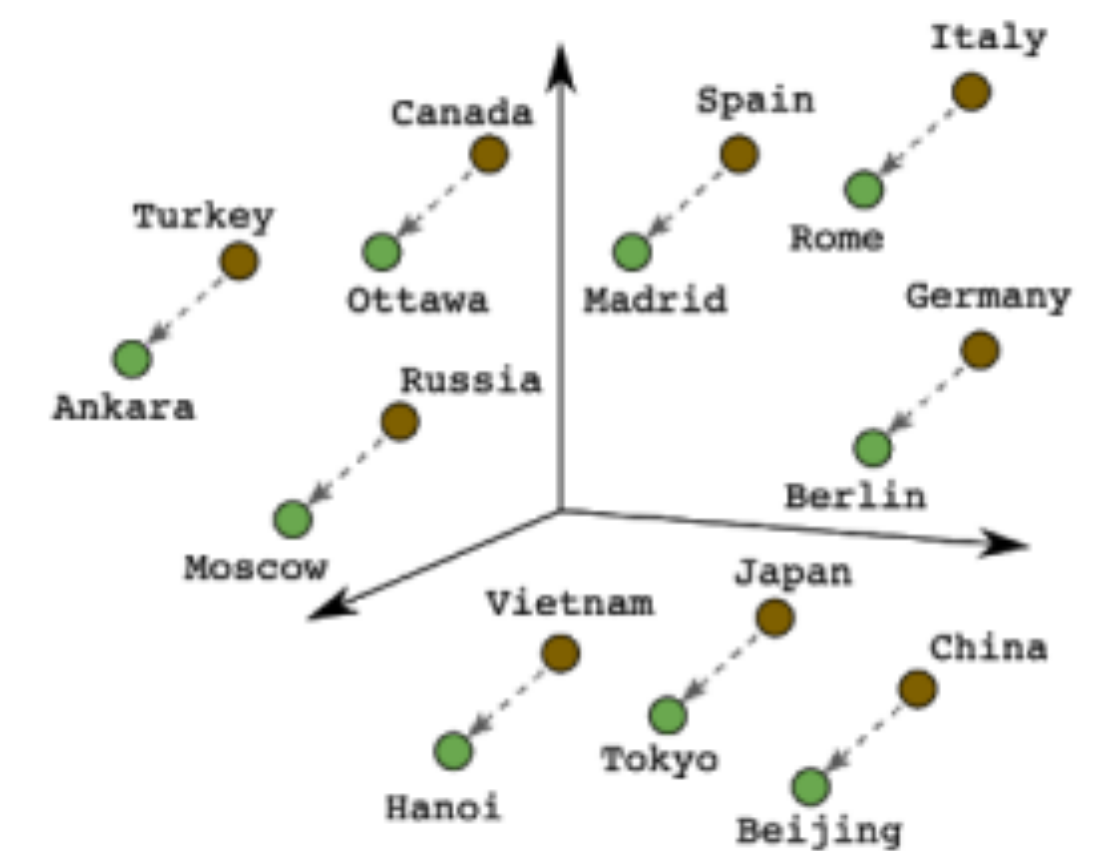
Una alternativa es proyectar los tokens en un **espacio vectorial continuo** donde las palabras con significados similares se **encuentren ubicada cerca en el espacio vectorial**. Por ejemplo, perro y gato deberían estar próximos entre sí y ambas alejadas de casa.



Male-Female



Verb Tense

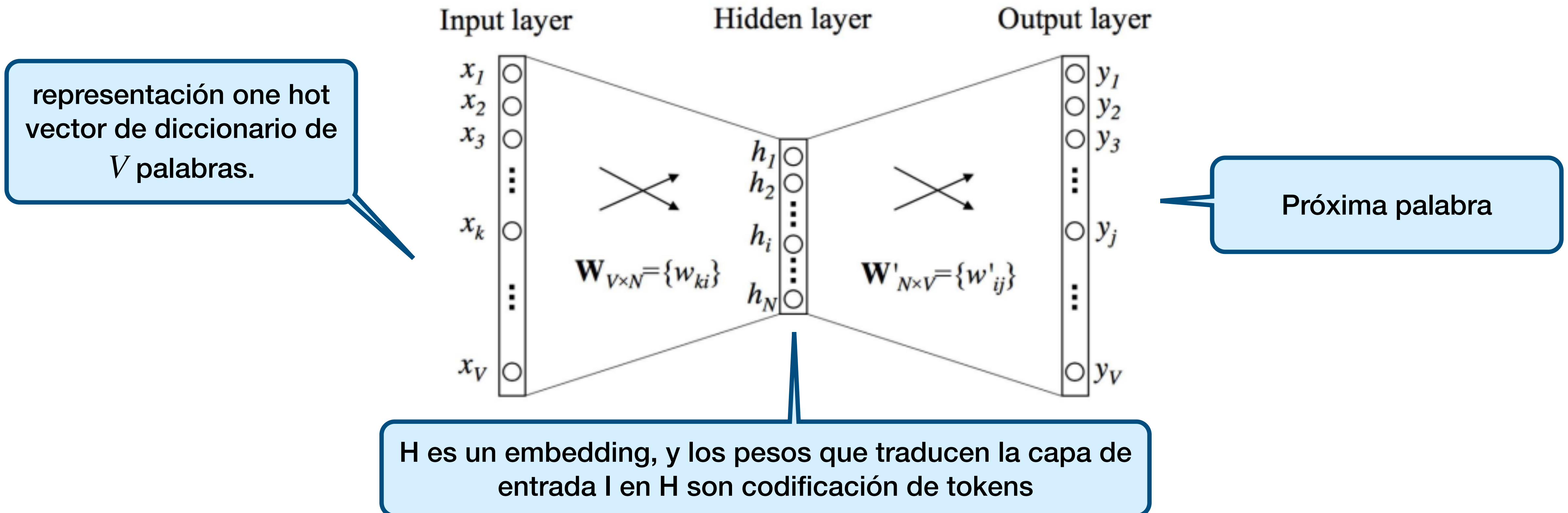


Country-Capital

¿ Cómo podemos calcular automáticamente estos valores ?

NLP - Dando semántica a los tokens = Embeddings

Una forma es entrenando una red neuronal que intente predecir la próxima palabra o computar análisis de sentimiento, etc. Si los pesos asociados a la capa oculta ayudan a responder las salidas correctas, de alguna manera sus valores capturan las relaciones entre las palabras.

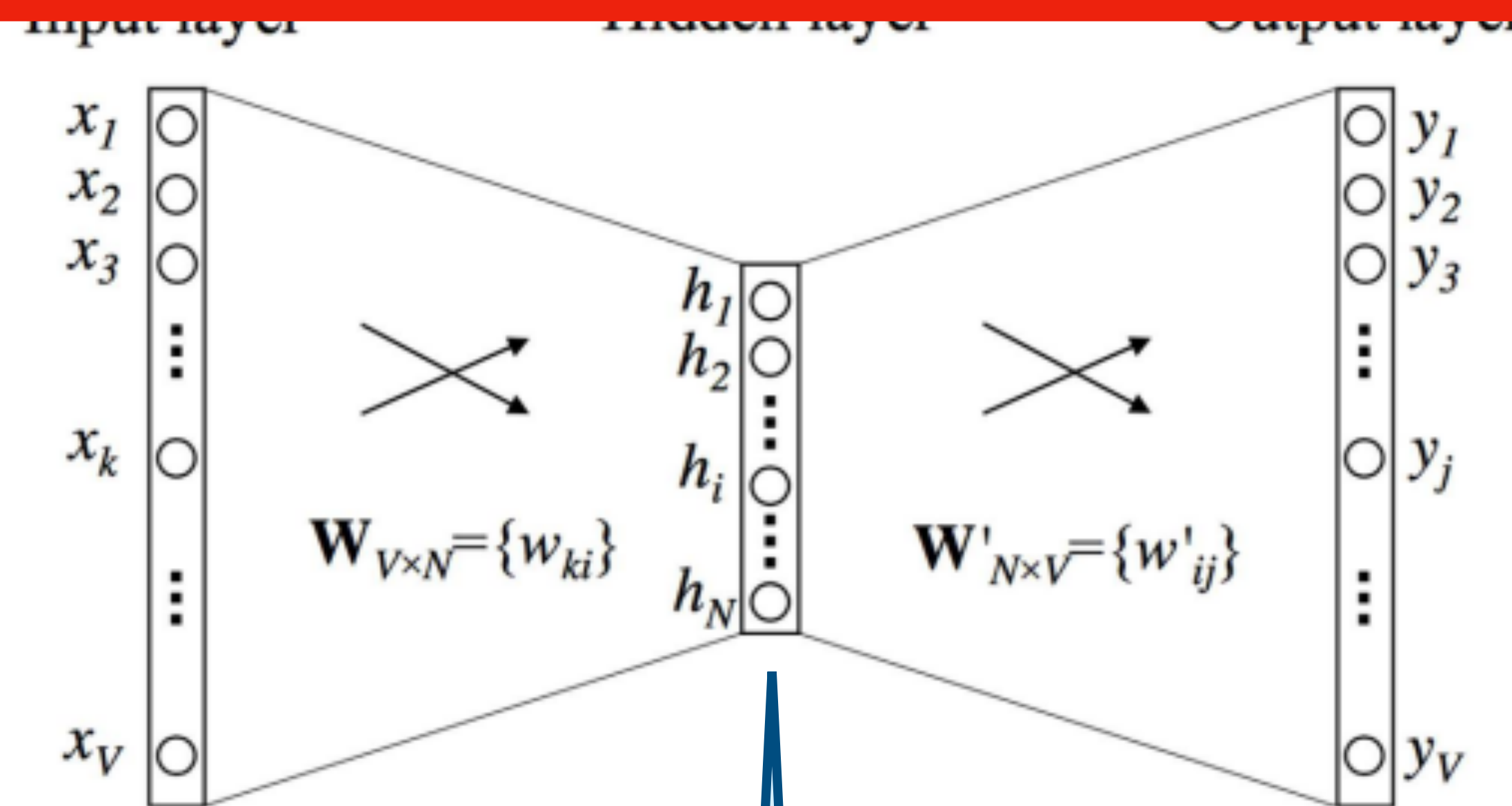


NLP - Dando semántica a los tokens = Embeddings

Una forma es entrenando una red neuronal que intente predecir la próxima palabra o computar análisis de sentimiento, etc. Si los pesos asociados a la capa oculta a una manera sus valores captu

Esta técnica se denomina Word2Vec, existen otras como GloVe que se basan en estadísticas globales sobre todo el corpus

representación one hot vector de diccionario de V palabras.



Próxima palabra

H es un embedding, y los pesos que traducen la capa de entrada I en H son codificación de tokens

NLP - Usando Embeddings

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from numpy.linalg import norm

text = "el gato duerme el gato corre el perro duerme el perro corre"
# Construimos el vocabulario
vocab = sorted(list(set(text.split())))
word_to_id = {w:i for i,w in enumerate(vocab)}
id_to_word = {i:w for w,i in word_to_id.items()}

print("Vocabulario:", word_to_id)
vocab_size = len(vocab)

# Convertimos el texto a IDs
encoded = np.array([word_to_id[w] for w in text.split()])

# Creamos pares de entrenamiento (input -> siguiente palabra)
X = encoded[:-1]
y = encoded[1:]

# 🛠 Ajustamos la forma de entrada: (num_samples, time_steps)
X = X.reshape(-1, 1) # cada entrada es una secuencia de longitud 1
y = y.reshape(-1, 1)

embed_dim = 8 # dimensión del embedding
hidden_units = 16 # neuronas LSTM

model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embed_dim),
    LSTM(hidden_units),
    Dense(vocab_size, activation='softmax')
])
```

```
model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam')
model.summary()

# entrenamos
model.fit(X, y, epochs=300, verbose=0)
print("Modelo entrenado")

# obtener embeddings
embeddings = model.layers[0].get_weights()[0]
print("Matriz de embeddings (shape):", embeddings.shape) # todos
print("\nEmbedding de la palabra 'gato':\n",
embeddings[word_to_id["gato"]])

# jugar jugando similitud
def simil(w1, w2):
    v1 = embeddings[word_to_id[w1]]
    v2 = embeddings[word_to_id[w2]]
    return np.dot(v1, v2) / (norm(v1) * norm(v2)) #producto escalar

print("Similitud(gato, perro):", simil("gato", "perro"))
print("Similitud(gato, duerme):", simil("gato", "duerme"))
```

NLP - Usando Embeddings

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from numpy.linalg import norm

text = "el gato duerme el gato corre el perro duerme el perro corre"
# Construimos el vocabulario
vocab = sorted(list(set(text.split())))
word_to_id = {w:i for i,w in enumerate(vocab)}
id_to_word = {i:w for w,i in word_to_id.items()}

print("Vocabulario:", word_to_id)
vocab_size = len(vocab)

# Convertimos el texto a IDs
encoded = np.array([word_to_id[w] for w in text.split()])

# Creamos pares de entrenamiento (input -> siguiente palabra)
X = encoded[:-1]
y = encoded[1:]

# 🔧 Ajustamos la forma de entrada: (num_samples, time_steps)
X = X.reshape(-1, 1) # cada entrada es una secuencia de longitud 1
y = y.reshape(-1, 1)

embed_dim = 8 # dimensión del embedding
hidden_units = 16 # neuronas LSTM

model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embed_dim),
    LSTM(hidden_units),
    Dense(vocab_size, activation='softmax')
])
```

```
model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam')
model.summary()

# entrenamos
model.fit(X, y, epochs=300, verbose=0)
print("Modelo entrenado")

# obtener embeddings
embeddings = model.layers[0].get_weights()[0]
print("Matriz de embeddings (shape):", embeddings.shape) # todos
print("\nEmbedding de la palabra 'gato':\n",
embeddings[word_to_id["gato"]])

# jugar probando similitud
def simil(w1, w2):
    v1 = embeddings[word_to_id[w1]]
    v2 = embeddings[word_to_id[w2]]
    return np.dot(v1, v2) / (norm(v1) * norm(v2)) #producto escalar

print("Similitud(gato, perro):", simil("gato", "perro"))
print("Similitud(gato, duerme):", simil("gato", "duerme"))
```

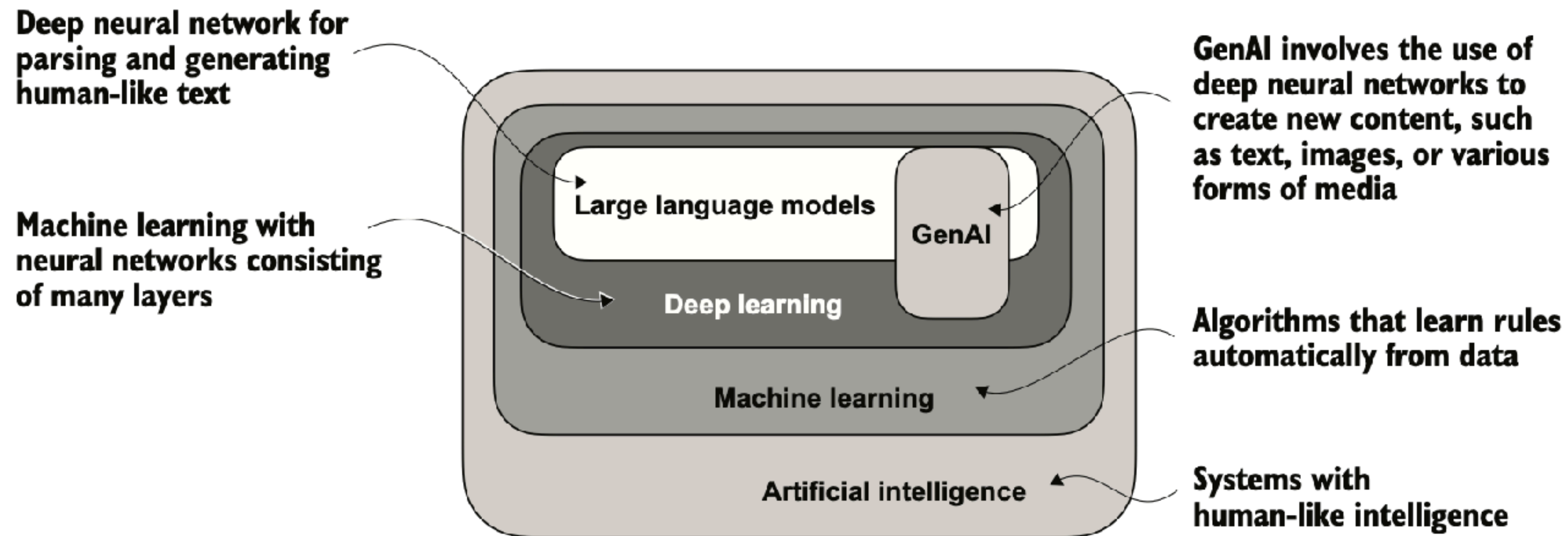
Podemos utilizar Embeddings pre-entrenados como Word2Vec, GloVe o FastText

NLP - Usando Embeddings - Similitud

Nombre	Fórmula (sin detalles)	Intuición	Valores típicos
Cosine Similarity	Ángulo entre vectores	Compara dirección, no magnitud	-1 a 1
Dot Product	Multiplicación vectorial	Más grande si apuntan al mismo sentido	No acotado
Euclidean Distance	Distancia geométrica	Más pequeño significa más similar	0 a ∞
Manhattan Distance	Distancia por sumas de diferencias	Útil en espacios dispersos	0 a ∞
Jaccard Similarity	Intersección / Unión	Similitud basada en conjuntos	0 a 1
Pearson Correlation	Correlación lineal	Mide relación lineal entre vectores	-1 a 1

NLP - Modelos de Lenguajes "Gigantes" Pre-entrenados (LLMs)

Un LLM es un modelo de inteligencia artificial basado en redes neuronales profundas, entrenado con enormes cantidades de texto para poder comprender y producir lenguaje humano de forma coherente y contextual.



NLP - Modelos de Lenguajes "Gigantes" Pre-entrenados (LLMs)

La mayoría de los LLM modernos se basan en la arquitectura **Transformer**, que es una arquitectura de red neuronal profunda introducida en el artículo de 2017 “Attention Is All You Need”.

Un componente clave de los transformers y los LLM es el mecanismo de auto-atención, que permite que el modelo evalúe la importancia relativa de diferentes palabras (o tokens) dentro de una secuencia.

Este mecanismo hace posible que el modelo **capture dependencias de largo alcance y relaciones contextuales en los datos de entrada**, mejorando su capacidad para generar resultados coherentes y relevantes según el contexto.

NLP - Modelos de Lenguajes "Gigantes" Pre-entrenados (LLMs)

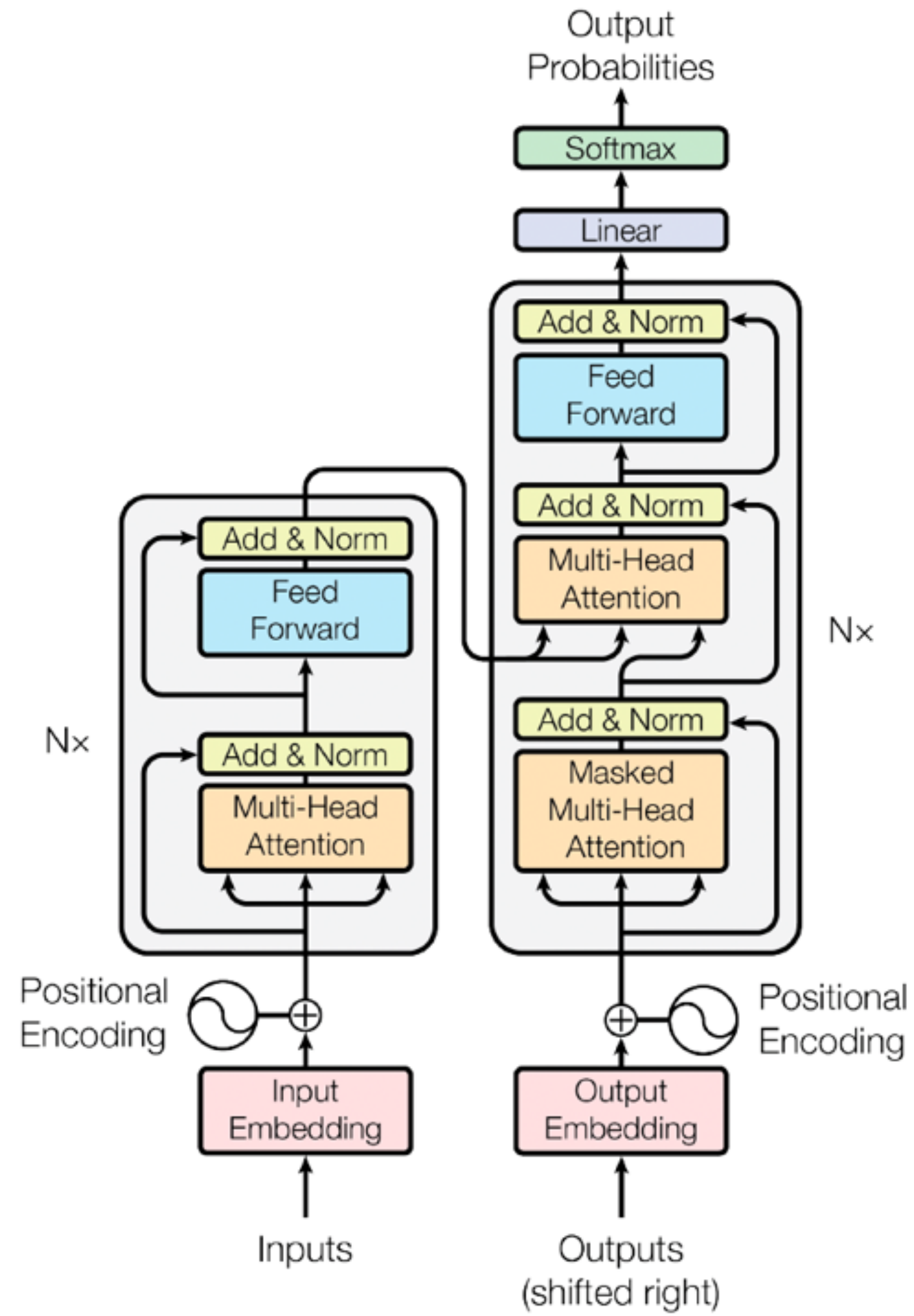
La mayoría de los LLM modernos se basan en la arquitectura **Transformer**, que es una arquitectura de red neuronal profunda introducida en el artículo de 2017 “Attention Is All You Need”.

Un componente clave de los transformers y los LLM es el mecanismo de auto-atención, que permite que el modelo evalúe la importancia relativa de diferentes palabras (o tokens) dentro de una secuencia.

Este mecanismo hace posible que el modelo **capture dependencias de largo alcance y relaciones contextuales en los datos de entrada**, mejorando su capacidad para generar resultados coherentes y relevantes según el contexto.

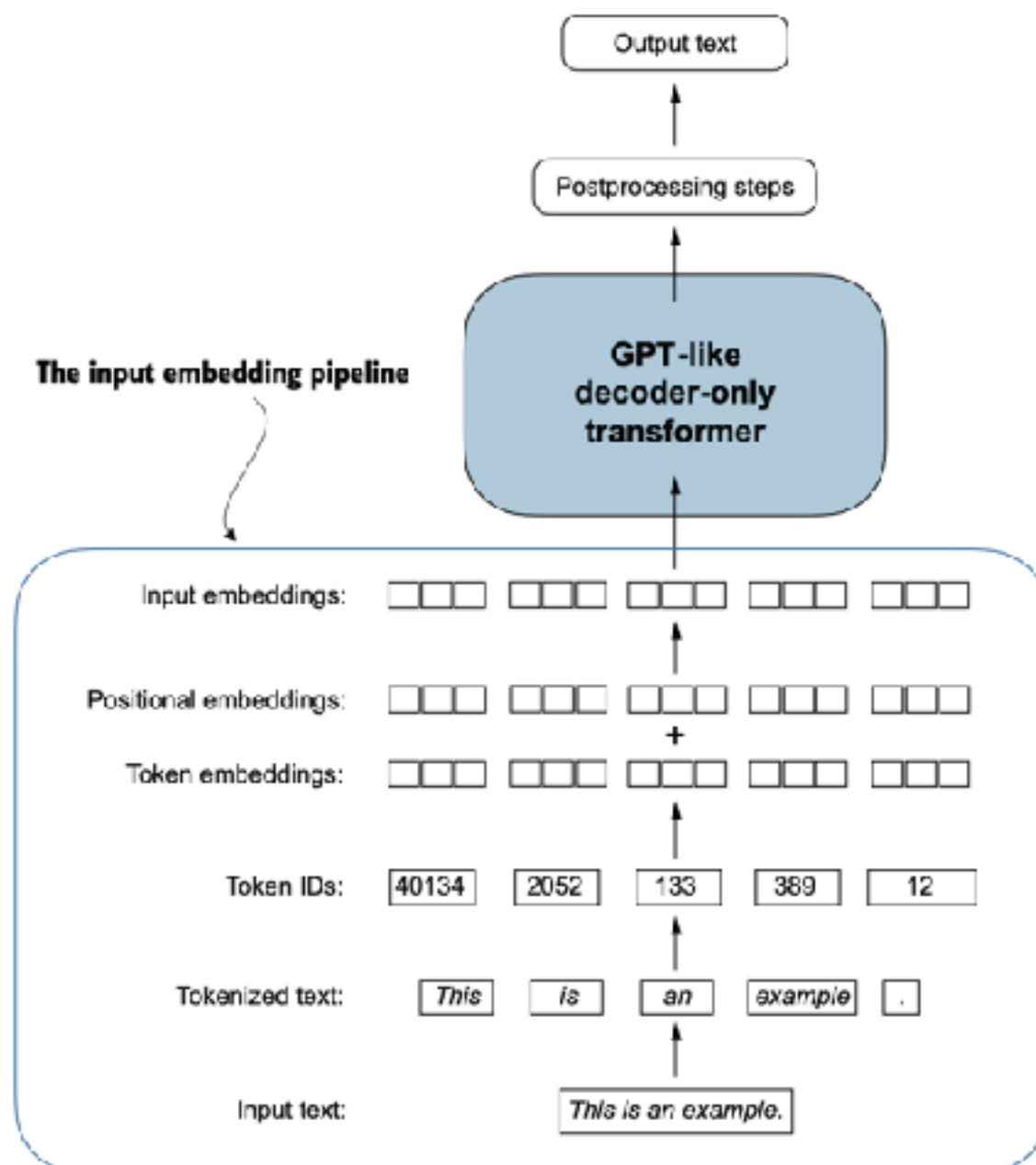
No todo Transformer es un LLM, se puede utilizar para generar otro tipo de información, por ejemplo imágenes

NLP - Transformers



NLP - Positional Embeddings

En la arquitectura Transformers, además del embedding de cada palabra, se necesita la posición de la misma dentro de la frase (imagen).



- **Positional Embeddings Fijos** (Sinusoidales) “Attention is All You Need”: Se generan usando funciones seno y coseno con frecuencias diferentes para cada dimensión del embedding.
- **Positional Embeddings Aprendidos** (Learned Positional Embeddings): Cada posición se trata como una palabra adicional en la tabla de embeddings.
- **Relative Positional Embeddings**: En vez de codificar la posición absoluta, codifican la distancia entre tokens. No importa dónde están los tokens, sino qué tan lejos están entre sí.
- **Rotary Positional Embeddings** (RoPE): Se rota cada embedding en el espacio geométrico de acuerdo con la posición. Esto permite que la matriz de atención capture desplazamientos.
- **ALiBi (Attention Linear Bias)**: Usado en modelos para inferencia muy rápida. No usa embeddings, sino que agrega un sesgo lineal a la atención

NLP - Transformers (Atención)

El mecanismo de atención permite que un modelo, cuando procesa una palabra, pueda mirar (o “prestar atención” a) otras palabras de la secuencia para decidir cuáles son más relevantes para entender su significado. En lugar de procesar todo por igual, el modelo calcula qué partes del contexto importan más en cada paso.

Esta técnica se base en la construcción y aprendizaje de 3 matrices que indican:

- **Query (Q)**: Lo que esta palabra busca “¿Qué información necesito?”
- **Key (K)**: Lo que esta palabra ofrece “¿Cuál es mi significado/contexto?”
- **Value (V)**: La información real que se comparte “Aporte.”

Se compara Query contra Key midiendo la relevancia (similitud) y se pondera (softmax). Luego, estas ponderaciones se aplican a los Value retornando una combinación contextualizada.

NLP - Transformers (Atención)



$palabra_1$

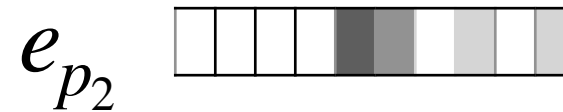
$palabra_2$

$palabra_n$

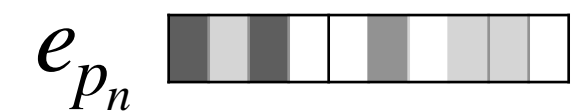
NLP - Transformers (Atención)



palabra₁

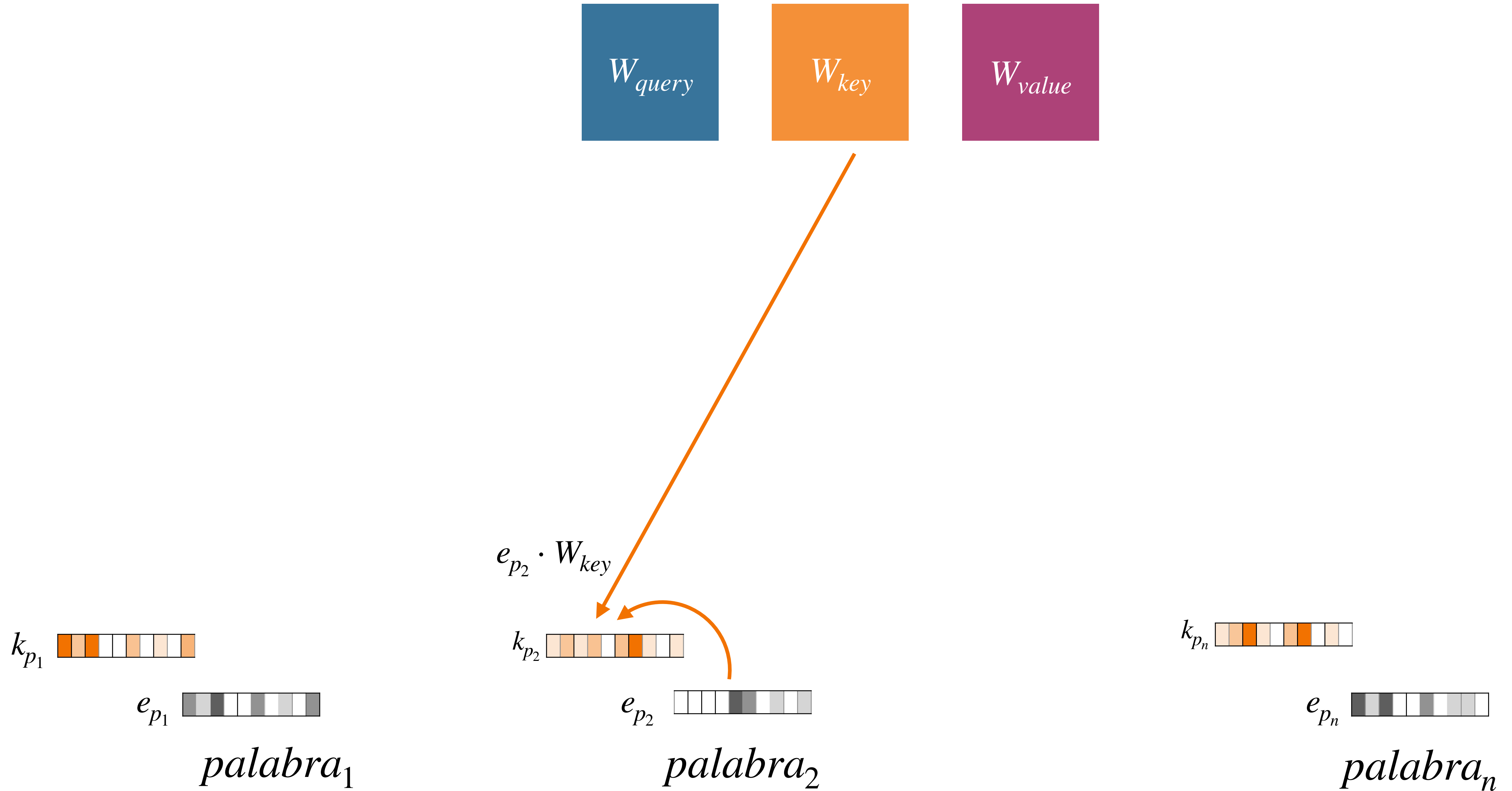


palabra₂

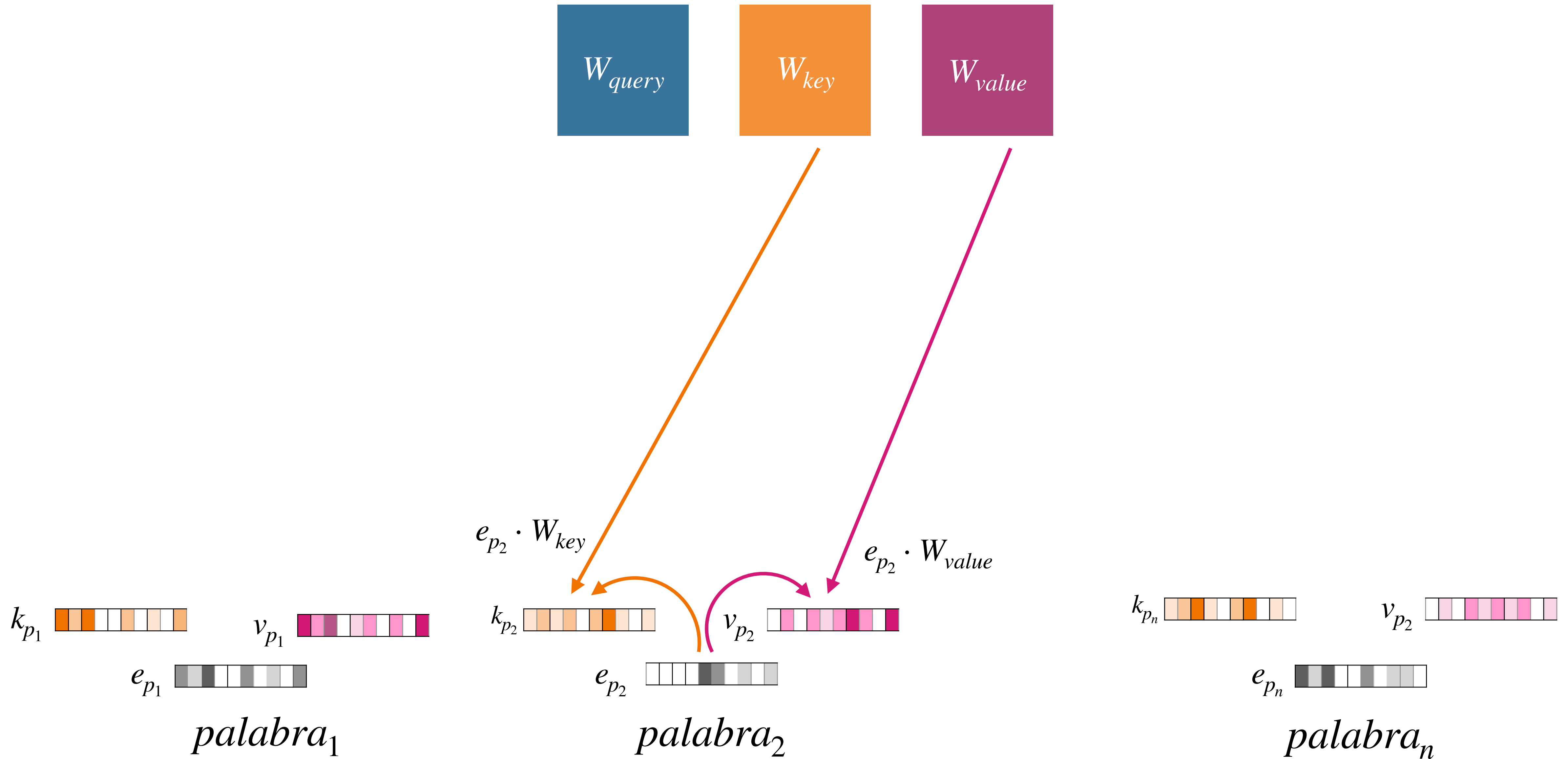


palabra_n

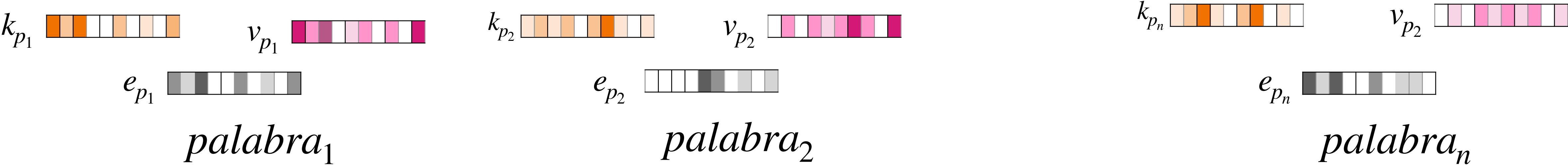
NLP - Transformers (Atención)



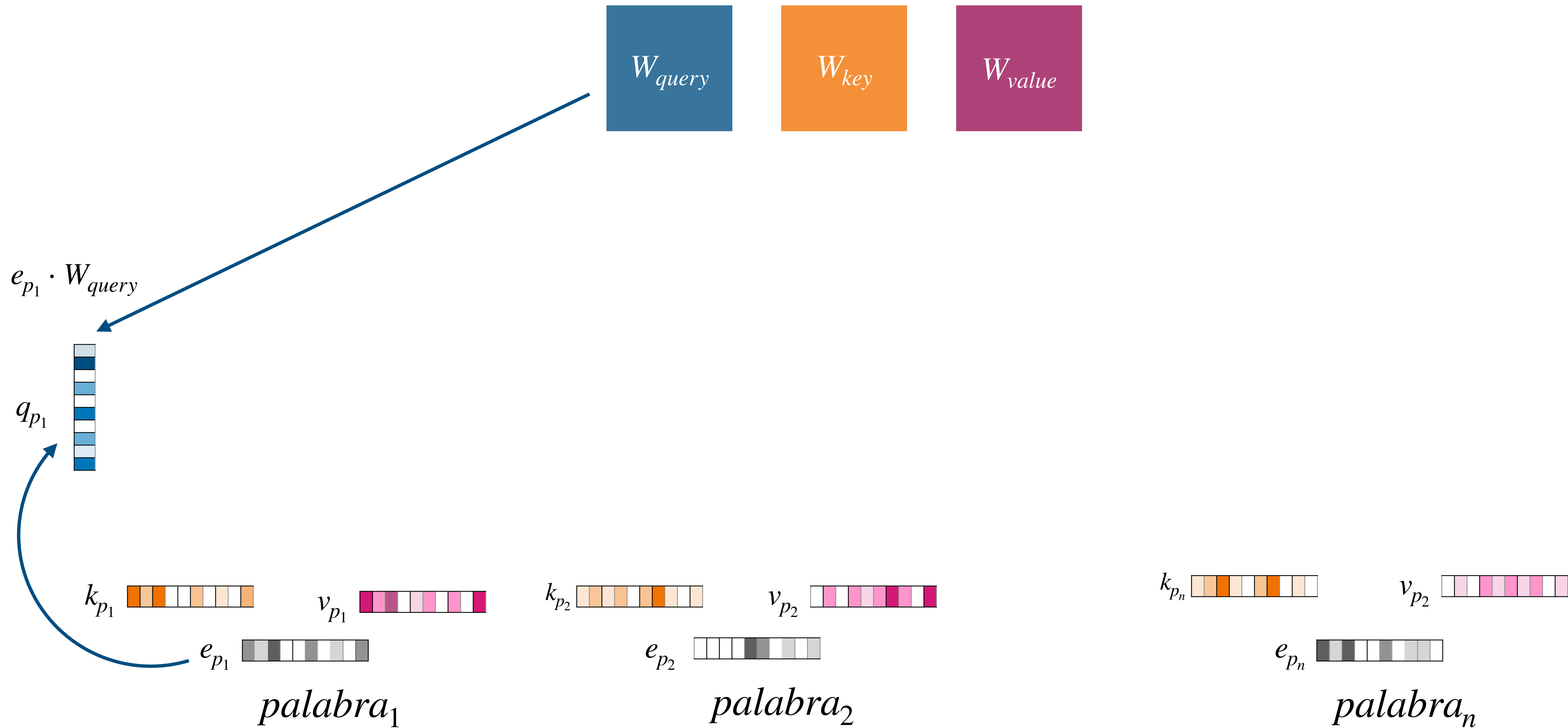
NLP - Transformers (Atención)



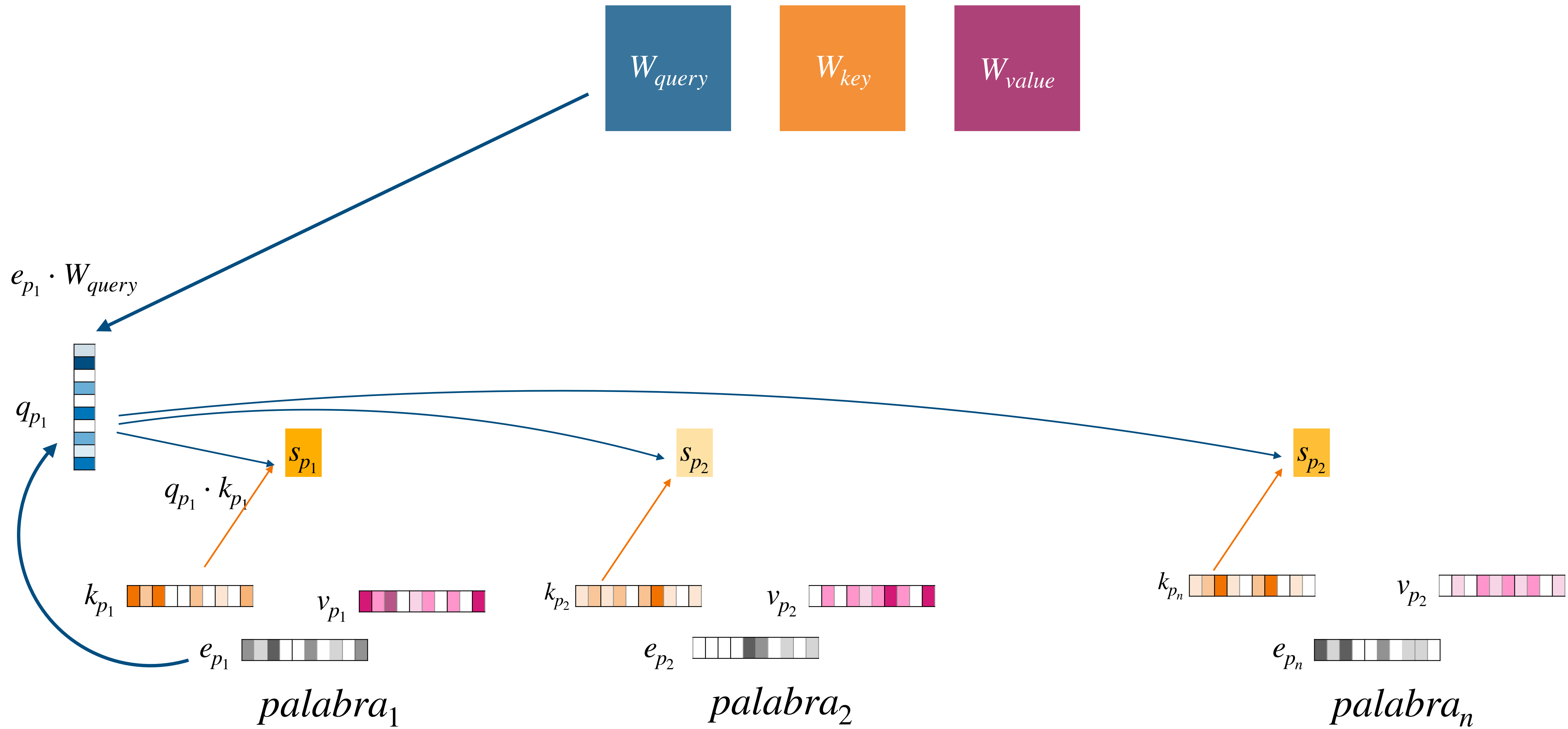
NLP - Transformers (Atención)



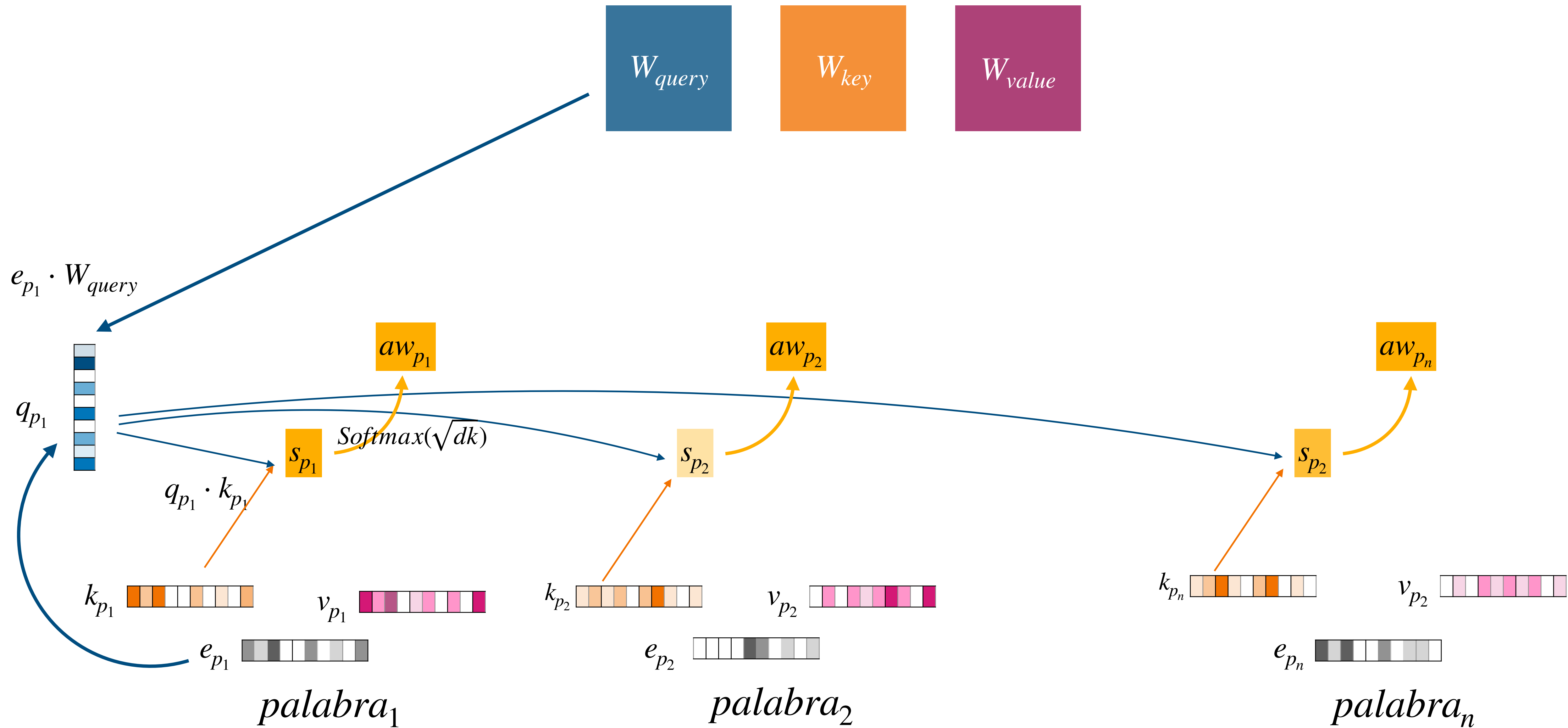
NLP - Transformers (Atención)



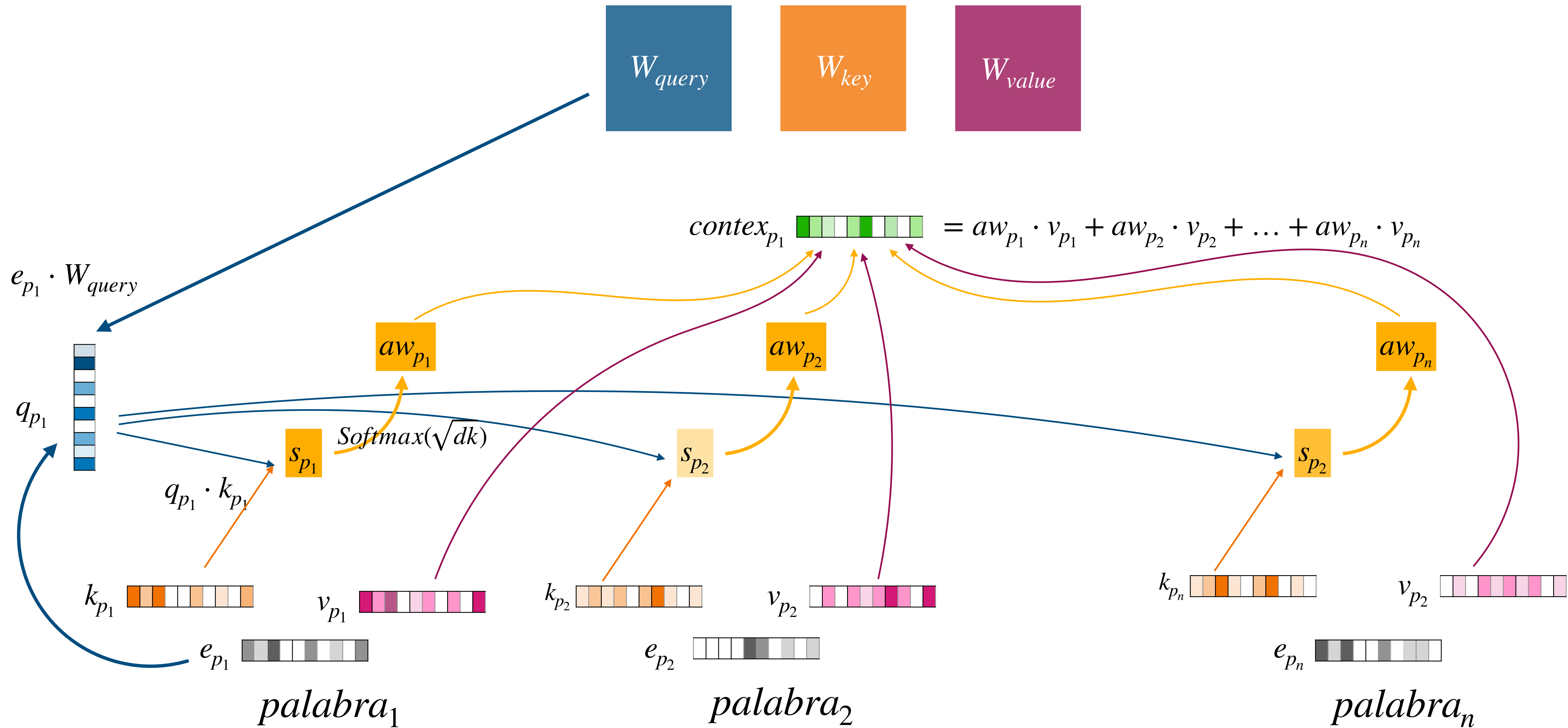
NLP - Transformers (Atención)



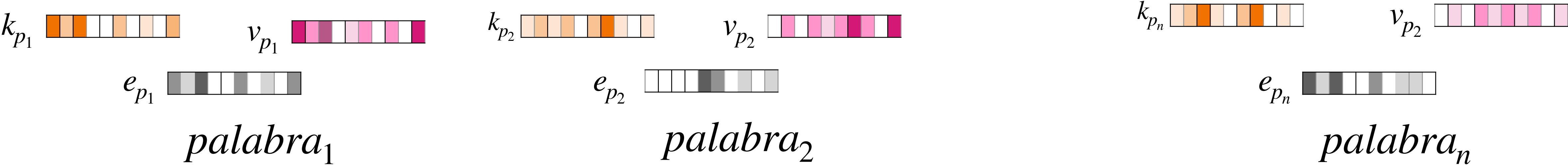
NLP - Transformers (Atención)



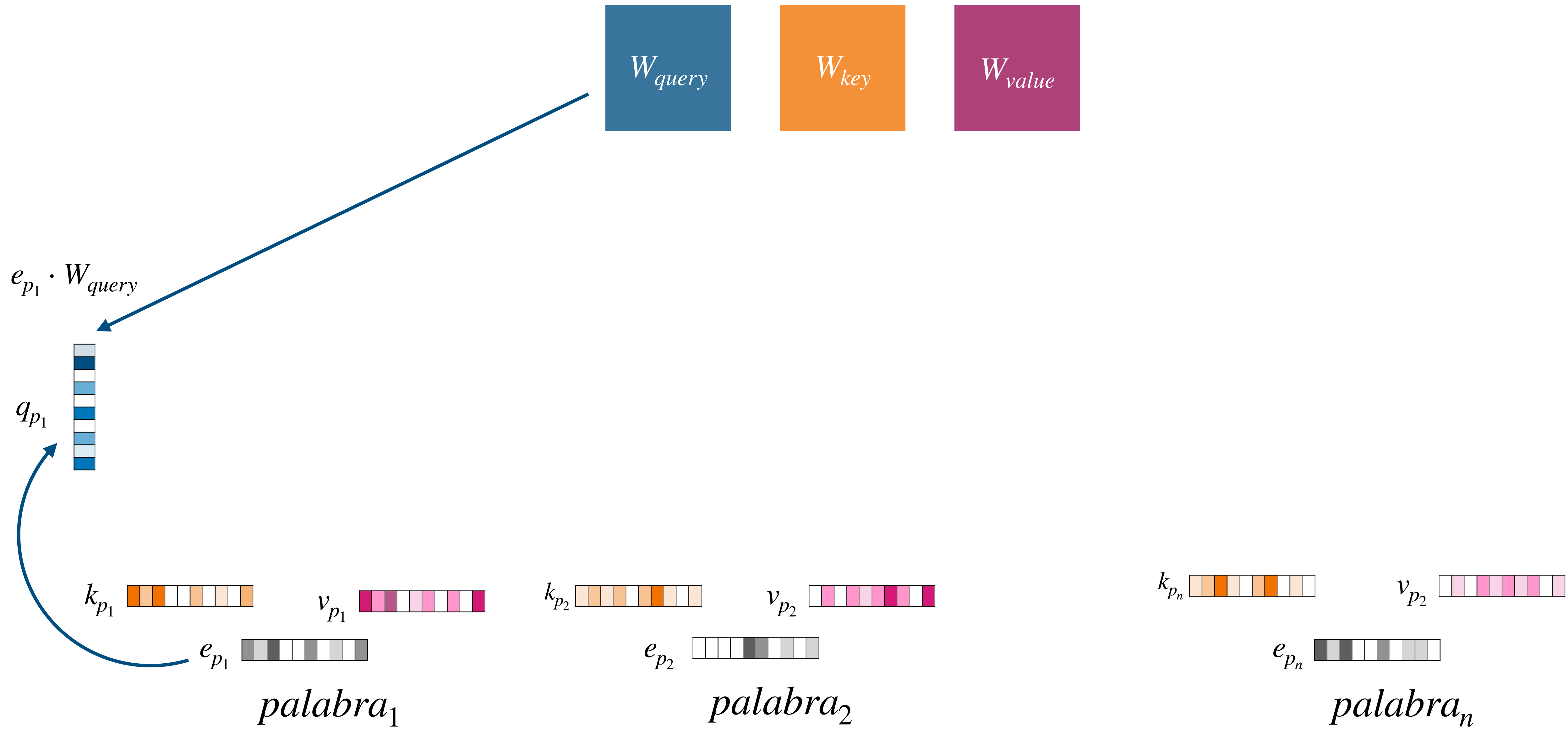
NLP - Transformers (Atención)



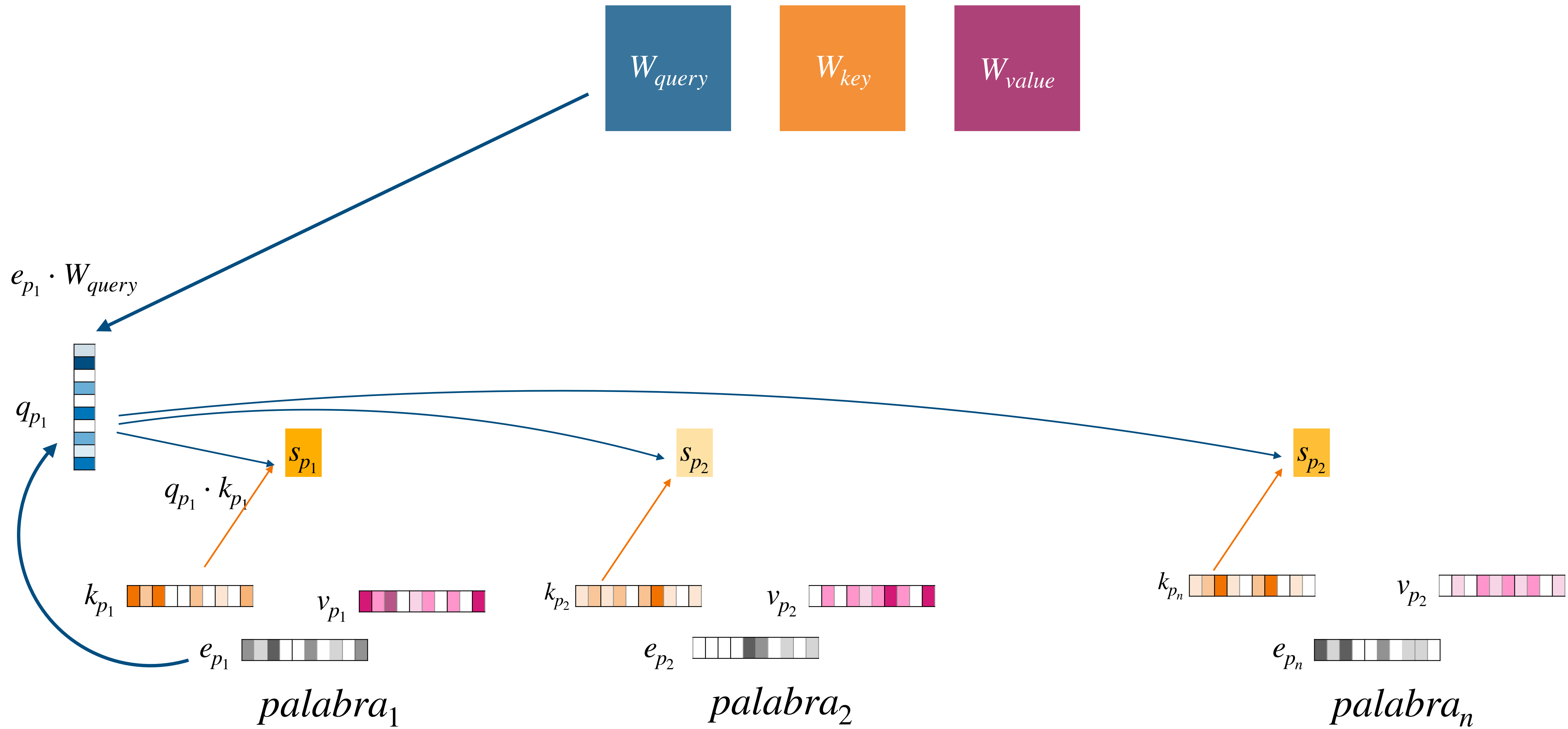
NLP - Transformers (Atención)



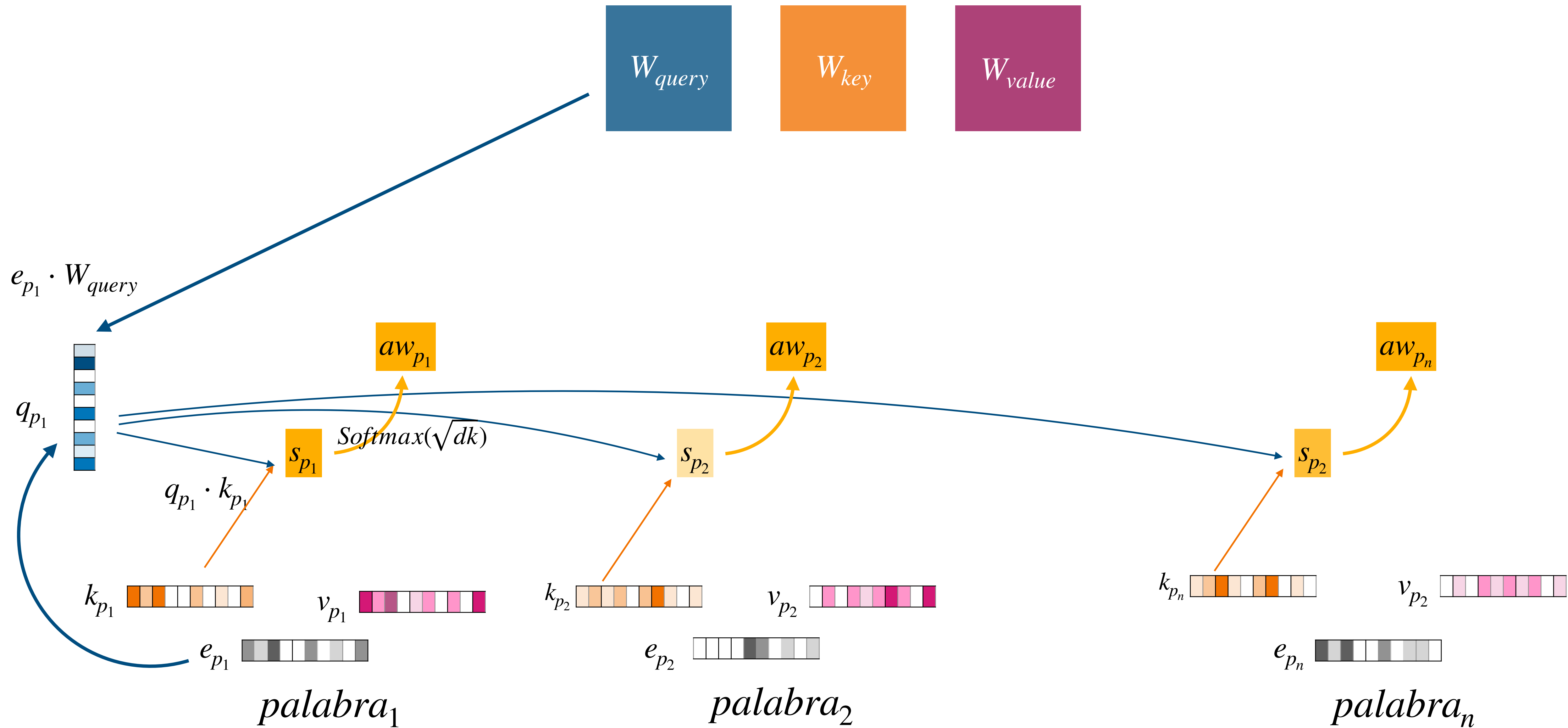
NLP - Transformers (Atención)



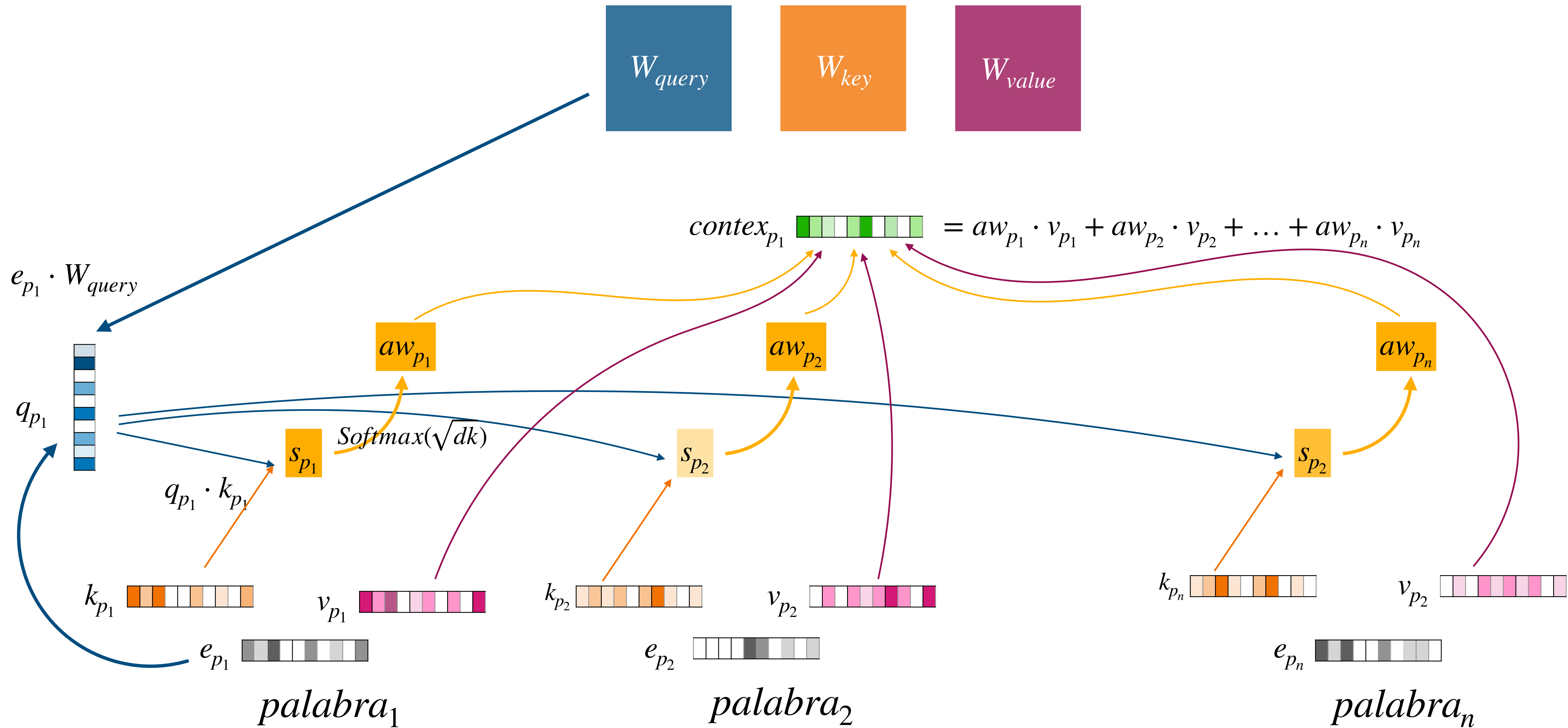
NLP - Transformers (Atención)



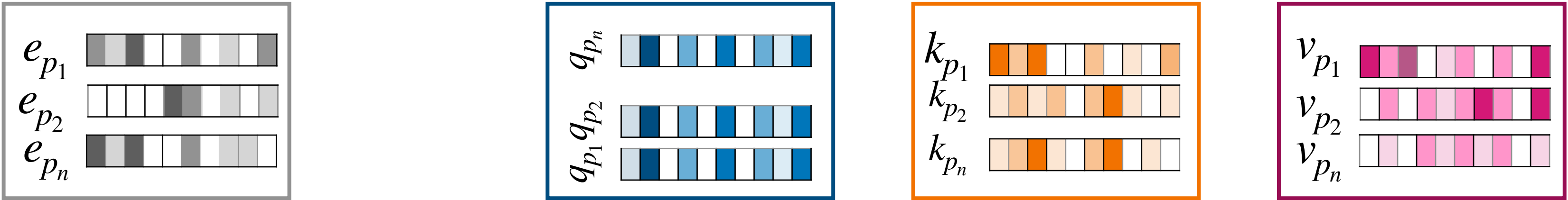
NLP - Transformers (Atención)



NLP - Transformers (Atención)

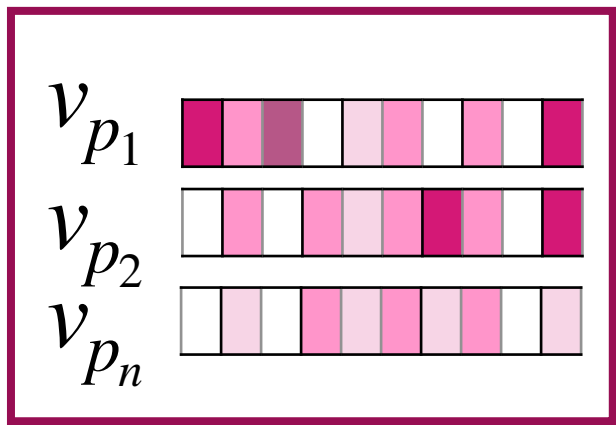
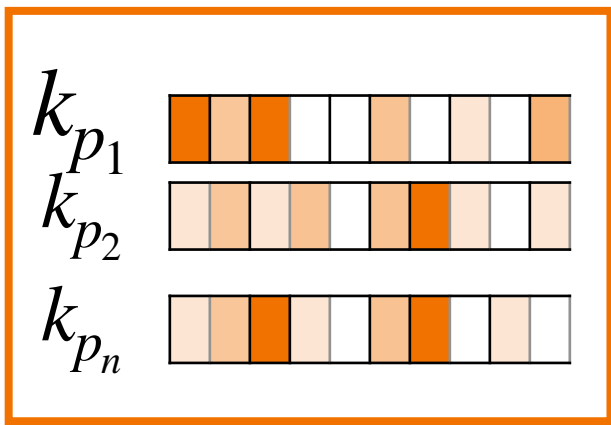
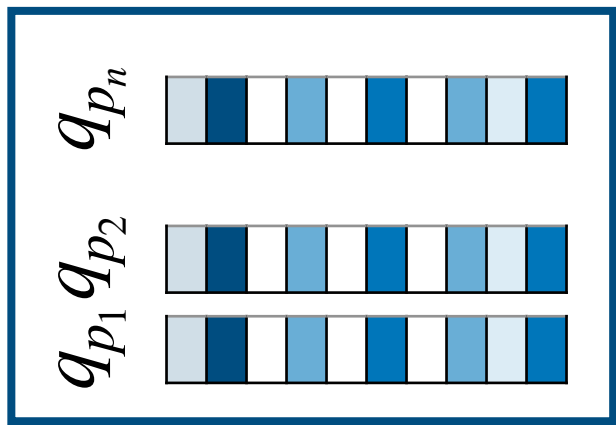
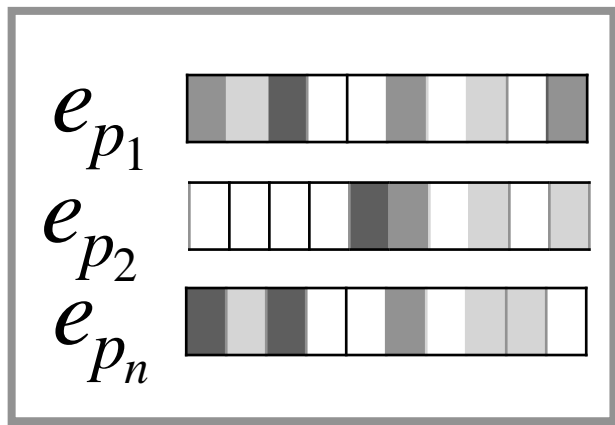


NLP - Transformers (Atención) - optimización



$$\text{Softmax} \left(\frac{\begin{matrix} \boxed{Q} & \boxed{K^T} \end{matrix}}{\sqrt{dk}} \right) \cdot \boxed{V} = \boxed{\text{Context}}$$

NLP - Transformers - Multi-Head-attention



$$\text{Softmax} \left(\frac{\begin{matrix} \text{blue box } Q & \text{orange box } K^T \end{matrix}}{\sqrt{dk}} \right) \cdot \begin{matrix} \text{purple box } V \end{matrix} = \begin{matrix} \text{green box } Context \end{matrix}$$

NLP - Self-Attention vs Cross-Attention

Self-attention permite que cada elemento de una secuencia se compare y relacione con los demás elementos **de la misma secuencia**, capturando dependencias internas y contexto global (por ejemplo, cómo las palabras de una oración se influyen entre sí).

Cross-attention permite que una secuencia (generalmente en el *decoder*) **se relacione con otra secuencia diferente** (proveniente del *encoder*), de modo que el modelo pueda usar la información previamente codificada; esto es esencial en tareas como traducción automática, donde el decoder genera la salida mirando lo que el encoder entendió de la entrada.

NLP - Masking y Dropout

En **Transformers**, el **masking** se utiliza para controlar qué información puede ser vista durante el cálculo de la atención. Por ejemplo, en modelos generativos (como los decoders de lenguaje), se aplica un *mask* que bloquea las palabras futuras para que el modelo no “vea” lo que aún no debe predecir, garantizando que la generación sea paso a paso y causal.

Dropout es una técnica de regularización que consiste en apagar aleatoriamente algunas neuronas durante el entrenamiento para evitar que el modelo dependa demasiado de ciertos patrones específicos; esto ayuda a prevenir el **overfitting** y mejora la capacidad de generalización.

Ambas técnicas trabajan juntas: el *masking* preserva la estructura y coherencia temporal de la información, mientras que el *dropout* controla el sobreajuste durante el aprendizaje.

NLP - Modelos de Lenguajes "Gigantes" Pre-entrenados (LLMs)

